

ICS-344: Information Security

DVSA Vulnerability Discovery and Remediation

Lesson #1 —Event Injection

Course / Section	ICS-344: Information Security — Term 252
Student Name	Moath Haimour
DVSA Website URL	http://dvsa-website-962415228810-us-east-1.s3-website.us-east-1.amazonaws.com/ AWS Region: us-east-1
API Gateway Endpoint	https://6lm7gmtzfg.execute-api.us-east-1.amazonaws.com/dvsa/order
AWS Region	us-east-1
Date	May 1, 2026
Vulnerability Title	Lesson #1 — Event Injection

Lesson #1: Event Injection

Part 1) Goal and Vulnerability Summary

Event Injection is a serverless security vulnerability where attacker-controlled input is executed as code instead of being treated as data. In this case, the vulnerability affects the AWS Lambda function behind the API Gateway /order endpoint in the DVSA-ORDER-MANAGER Lambda function, which processes user requests. The security impact is severe because an attacker can inject and execute arbitrary code, potentially invoking other Lambda functions or accessing sensitive data. The main weakness is the use of insecure deserialization (node-serialize) combined with a lack of input validation, allowing malicious payloads to be executed. This results in a form of remote code execution (RCE) within the serverless backend.

Part 2) Why This Works / Root Cause

The vulnerability exists because the application uses **unsafe deserialization** with the node-serialize library on user-controlled input. This allows specially crafted payloads such as `__$ND_FUNC$$_function(){...}()` to be interpreted and executed as code instead of being treated as data. Additionally, there is no strict validation of the action parameter, so malicious input is not blocked before processing. As a result, the backend executes attacker-controlled code inside the Lambda function, leading to remote code execution.

Part 3) Environment and Setup

Environment and Setup

- **DVSA API endpoint used:**
<https://6lm7gmtzfg.execute-api.us-east-1.amazonaws.com/dvsa/order>
- **AWS services involved:**
Amazon API Gateway, AWS Lambda (order-manager function)
- **User context used:**
Authenticated user with valid JWT token obtained from the DVSA application
- **Tools used:**
Postman, Browser DevTools (Network tab), AWS Console (for code inspection)

Part 4) Reproduction Steps

Reproduction Steps

1. **Access the DVSA application**
Open the DVSA website in the browser and perform a normal action (e.g., place an order) to generate a request to the backend.
2. **Capture the API request**
Open Browser DevTools → Network tab and locate the request sent to:

POST /dvsa/order

3. **Copy the request to Postman**

Copy the request (headers and body) and recreate it in Postman.

Include the Authorization header with a valid JWT token (redacted in the report).

4. **Modify the request payload**

Replace the normal action value with the malicious payload:

```
__$ND_FUNC__$_(){var aws=require("aws-sdk");var lambda=new aws.Lambda();var p = {FunctionName:
"DVSA-ORDER-ORDERS", InvocationType: "RequestResponse", Payload: JSON.stringify({"user": " function
REDACTED-USER-ID"}});lambda.invoke(p,function(e,d){ return d; }); }
```

5. **Send the malicious request**

Submit the modified request to the API endpoint using Postman.

6. **Observe the response**

The server returns a **502 Internal Server Error**, indicating that the injected function was processed and caused a backend failure.

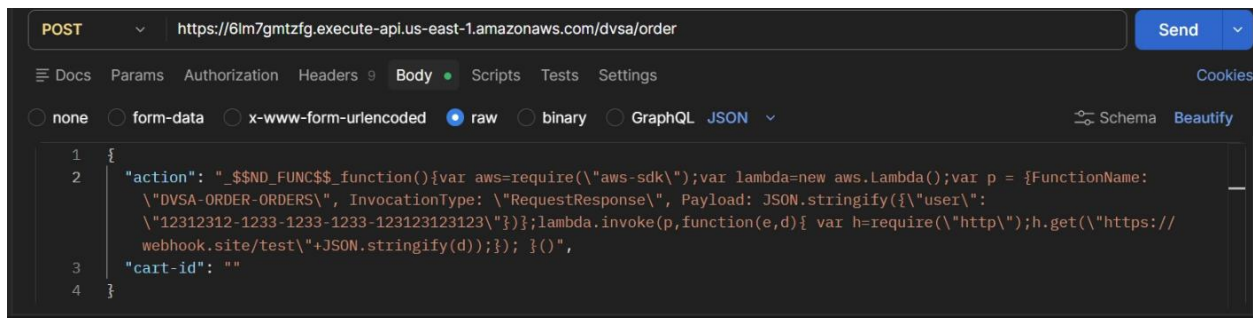
Part 5) Evidence and Proof

The vulnerability was demonstrated by sending a malicious payload to the /dvsa/order API endpoint using Postman.

The 502 Internal Server Error confirms that the backend Lambda function attempted to process the injected payload and failed during execution. This indicates that the input was interpreted as executable code instead of plain data, proving the presence of an Event Injection (code execution) vulnerability.

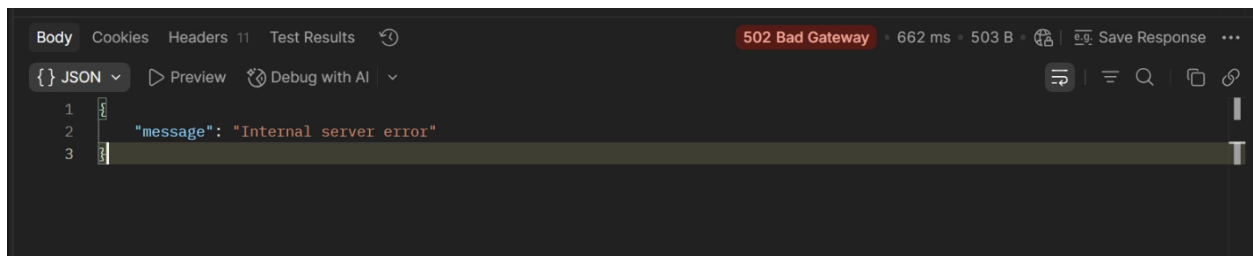
Evidence 1: Postman Request (Exploit Payload)

A modified request was sent where the action parameter contained the injected payload:

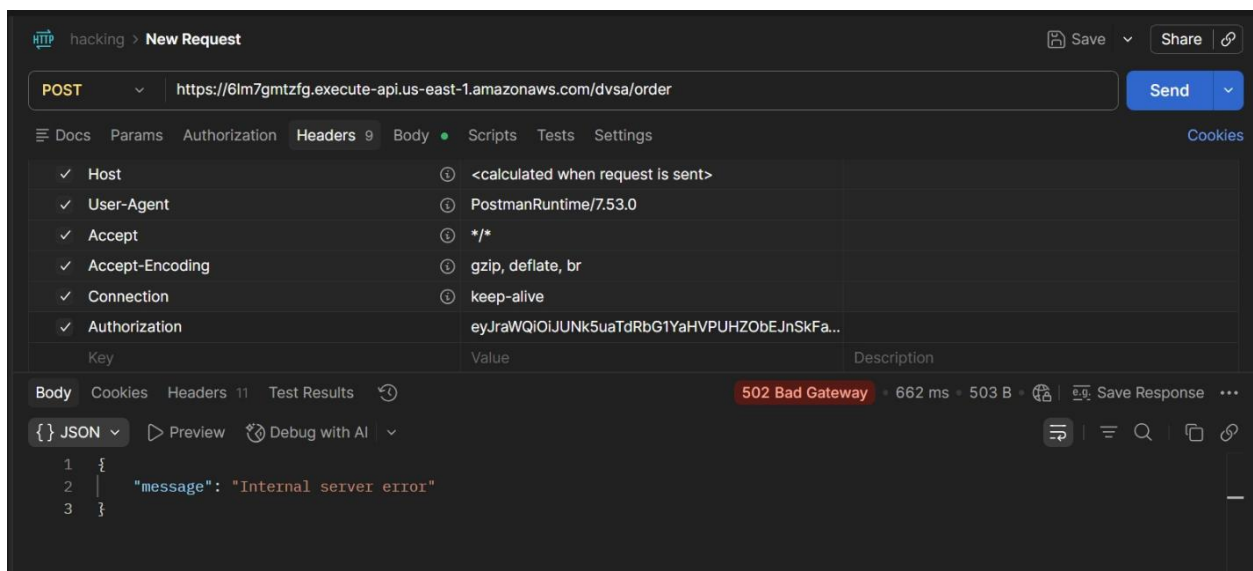
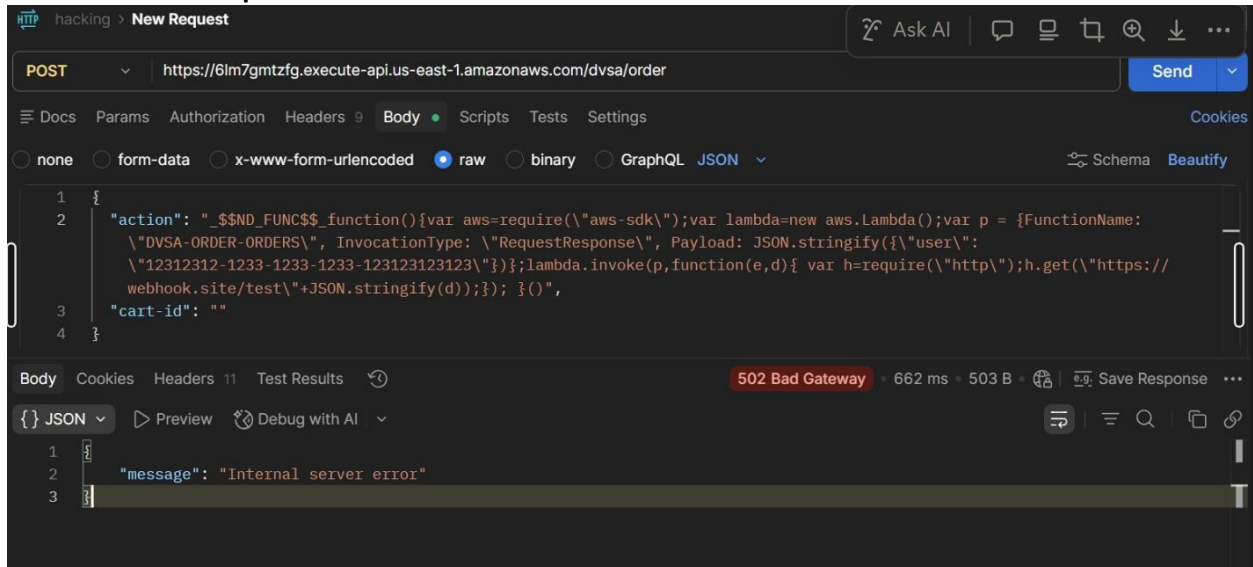


Evidence 2: Server Response

The server responded with:



Evidence 3: Full Request

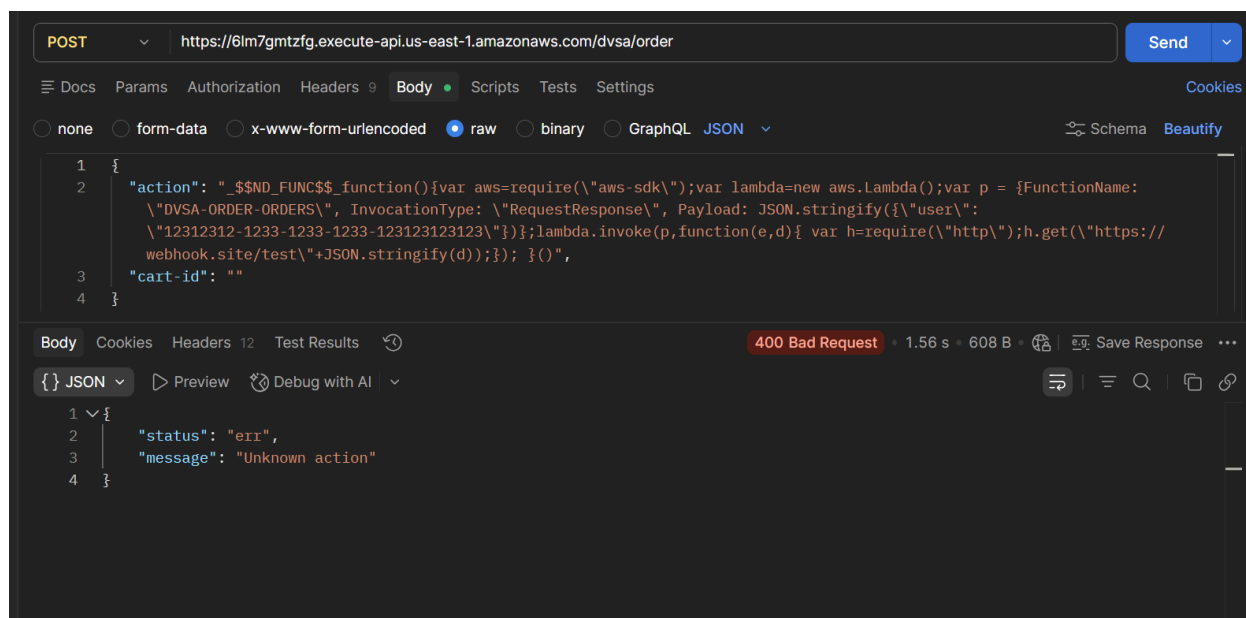


Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the **AWS Lambda function (order-manager)** that processes API Gateway requests. The vulnerability is caused by unsafe deserialization using the node-serialize library and lack of input validation.

To mitigate this, the insecure deserialization was removed and replaced with safe JSON parsing using `JSON.parse`, ensuring that user input is treated strictly as data and not executed as code. In addition, strict allow-list validation was implemented for the action parameter, allowing only predefined valid operations.

This security improvement prevents malicious payloads such as `__$ND_FUNC$$_function(){...}()` from being interpreted or executed by the backend. By eliminating dynamic execution and enforcing input validation, the fix directly addresses the root cause of the vulnerability and prevents remote code execution in the Lambda function.



Part 7) Code / Config Changes

AWS Resource changed:

DVSA-ORDER-MANAGER (AWS Lambda function)

File path:

backend/functions/order-manager/order-manager.js

Before (Vulnerable Code)

```

const serialize = require('node-serialize');

var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);

```

After (Fixed Code)

```

var req;
try {
  req = JSON.parse(event.body);
} catch (e) {

```

```

return callback(null, {
  statusCode: 400,
  headers: { "Access-Control-Allow-Origin": "*" },
  body: JSON.stringify({ status: "err", message: "Invalid request body" }),
});
}

var headers = event.headers || {};

```

Additional Security Validation Added

```

const ALLOWED_ACTIONS = new Set([
  "new", "update", "cancel", "get", "orders",
  "account", "profile", "shipping", "billing",
  "complete", "inbox", "message", "delete",
  "upload", "feedback", "admin-orders",
]);

if (!ALLOWED_ACTIONS.has(req.action)) {
  return callback(null, {
    statusCode: 400,
    headers: { "Access-Control-Allow-Origin": "*" },
    body: JSON.stringify({ status: "err", message: "Unknown action" }),
  });
}

```

What was changed

- Removed the node-serialize library, which allowed unsafe deserialization and execution of injected payloads
 - Replaced it with JSON.parse() to safely process user input as data
 - Added strict allow-list validation for the action parameter
 - Added error handling for invalid JSON input
-

Why this fixes the issue

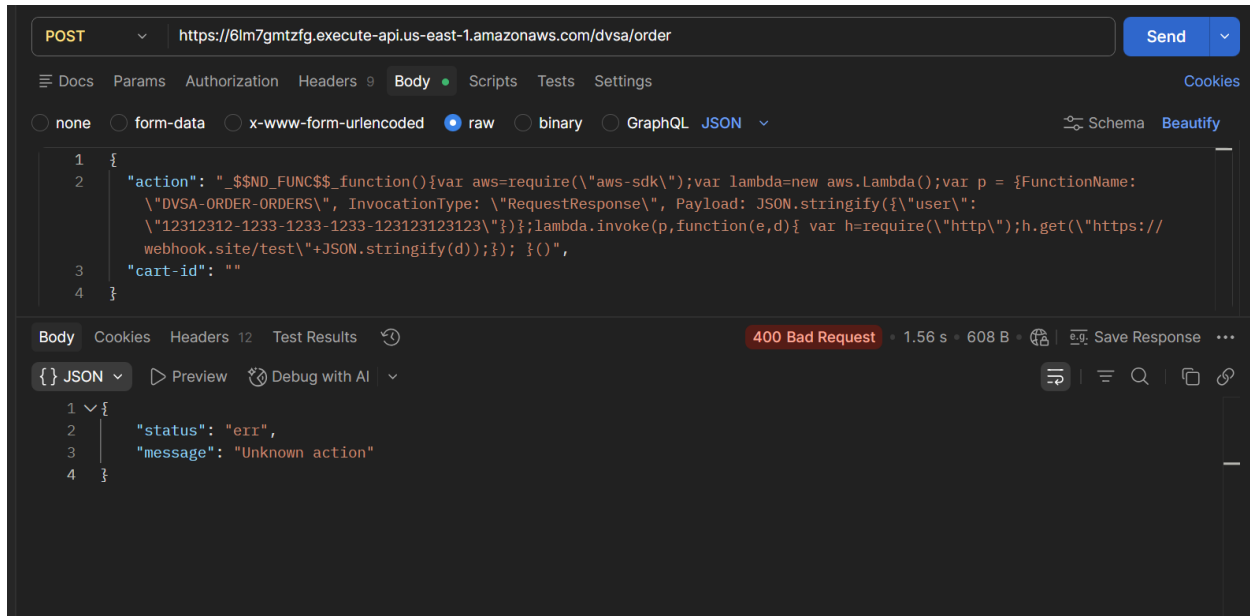
The original code allowed attacker-controlled input to be deserialized and executed as code. The updated implementation ensures that input is treated strictly as JSON data and blocks any unexpected or malicious values before they reach the application logic. This prevents execution of injected payloads such as `_$ND_FUNC$_function(){...}()`.

Part 8) Verification After Fix

Verification After Fix

Post-fix result:

After applying the fix, the same malicious payload was sent again using Postman. The server returned:

**What changed:**

The application no longer executes the injected payload because unsafe deserialization was removed and input validation blocks invalid actions.

Legitimate behavior:

Valid actions such as "new" and "orders" still work correctly, confirming that normal functionality is not affected.

Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 1: Event Injection	The action field must be treated as data only and must match predefined values. The backend must not execute user input.	Source code (order-manager.js), Postman requests/responses, DVSA lesson description	Sending "action": "new" returns a normal valid response	Sending <code>__\$ND_FUNC\$\$_function(){...}()</code> caused 502 Internal Server Error, showing code execution

Table B — Deviation analysis, fix, and verification

.

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson 1: Event Injection	The application executed user input as code due to unsafe deserialization, violating the rule that input must not be executed	Misuse	Removed node-serialize and added allow-list validation in order-manager.js (Lambda function)	Same payload now returns 400 Bad Request with "Unknown action" and is not executed	<i>[Optional timing result]</i>

9.1 Intended Logic and Security Rule(s)

The user sends a request from the DVSA frontend to the API Gateway endpoint `/dvsa/order`. The request is forwarded to the AWS Lambda function (order-manager), which processes the request based on the action field. The Lambda function should validate the request, ensure the user is authenticated, and then invoke the appropriate backend function. The expected outcome is that only valid actions are processed and user input is treated strictly as data.

Security rules:

- The action field must only contain predefined valid values
- User input must never be executed as code
- The backend must validate all request data before processing

9.2 Evidence Sources and Behavior Trace

Artifacts used:

- Source code (order-manager.js)
- Postman requests and responses
- DVSA lesson description
- API response screenshots

Normal behavior:

- Request with "action": "new" returns a valid response

Exploit behavior:

- Request with `__$ND_FUNC$_function(){...}()` caused 502 Internal Server Error, showing code execution

Post-fix behavior:

- Same payload returns 400 Bad Request with "Unknown action" and is not executed

9.3 Deviation Analysis and Classification

The exploit behavior violates the intended rule that user input must not be executed. The application used unsafe deserialization (node-serialize), which allowed attacker-controlled input to be interpreted as executable code. This is proven by the 502 Internal Server Error observed during the exploit. This behavior is incorrect because it allows arbitrary code execution in the backend.

Classification:

Intentional misuse / security-relevant abuse

9.4 Explainable Fix and Post-Fix Validation

The original logic incorrectly assumed that request data could be safely deserialized using node-serialize. The fix was applied in the Lambda function (order-manager.js) by removing unsafe deserialization and replacing it with JSON.parse, along with adding allow-list validation for the action field.

After the fix, the same malicious payload was tested and the server returned 400 Bad Request instead of executing the payload. This confirms that the vulnerability is resolved. Legitimate actions such as "new" continue to work correctly, showing that normal functionality is preserved.

Part 10) Takeaway / Lessons Learned

This vulnerability occurred because the application trusted user input and allowed it to be executed as code through unsafe deserialization. In a serverless environment, this can lead to serious impact such as invoking other Lambda functions or accessing sensitive data. The main lesson is to always treat user input as data, validate it strictly, and avoid any form of dynamic execution. Applying secure coding practices such as input validation and safe parsing can prevent similar vulnerabilities.